

Network Administration / System Administration

Homework 6

Student: 吳亞倫
ID: B13901011
Department: 電機工程學系

Discussion Partners:

- Classmates : 賴泓安、鄭竣陽、喻慈恩

1 Short Answers

1.1 Web server

References:

- [What is the difference between a static web server and a dynamic web server? - Quora](#)
- [系統設計 - 正向代理跟反向代理 - JYT0532](#)
- [用人類語言跟你說甚麼是正向代理和反向代理](#)
- [What is Load Balancing? - AWS](#)

Problem 1.1 (a) 以下為 Static Web Server 和 Dynamic Web Server 的差異：

- Static Web Server 是用來架設靜態網頁的伺服器，通常是用來提供靜態檔案，包含 HTML、CSS、JavaScript、圖片、影片等。它的主要功能是將這些靜態檔案傳送給客戶端，並不需要進行任何的運算或處理。常見的 Static Web Server 包含 Apache、Nginx、Microsoft IIS 等等。
- Dynamic Web Server 則是用來架設動態網頁的伺服器，他會根據 Client 端不同的請求，來進行一些後端的運算處理，並且將運算的結果傳送給 Client 端。這些運算處理通常是由後端的程式語言來完成，例如 PHP、Python、Ruby 等等。
- Static Web Server 特點：速度快、資源消耗少、容易擴展或部署到 CDN 上。
- Dynamic Web Server 特點：速度較慢、可以和資料庫或是 API 進行互動、可以根據使用者的請求來產生不同的內容。

Problem 1.1 (b)

- 正向代理 (Forward Proxy) 是指 Client 端向 Proxy Server 發送請求，然後由 Proxy Server 將請求轉發到目標伺服器，並將目標伺服器的回應傳回給 Client。正向代理通常用於隱藏 Client 的 IP 位址、過濾內容、加速網頁載入等。

- 反向代理 (Reverse Proxy) 是指有多台 Server 端，當 Client 發送請求時，Proxy Server 會根據請求的內容，將請求轉發到不同的 Server 上，並將 Server 的回應傳回給 Client。反向代理通常用於負載平衡、Caching 等。
- 使用 Proxy 的好處在於可以透過 Caching 等技術，來加快網路存取，並且減少 Server 的負擔。Proxy 也可以用來隱藏 Client 或 Server 的 IP 位址，並且可以過濾不必要的內容。

Problem 1.1 (c)

- 在現代的網路當中，伺服器的流量皆非常龐大，如果所有 Client 的請求都直接發送到 Server 上，會造成 Server 的負擔過重。因此，Load Balance 的概念就是將 Client 的請求分散到多台 Server 上，讓每台 Server 的負擔都能夠平均分配。這樣不僅可以提高 Server 的效能，還可以避免單一 Server 故障造成整個系統的癱瘓。
- 常見的 Load Balance 方法包含 Round-robin、Least Connections、IP Hash 等等。

1.2 C10k

References:

- [Apache vs. Nginx A Definitive Comparison](#)
- [Nginx Architecture and Why Can Nginx Handle Thousands of Requests Concurrently](#)
- [Why does Nginx starts process as root? - Unix Stack Exchange](#)

Problem 1.2 (a)

- Apache 的 process-based architecture 使用的是一個多執行緒的方式，當收到一筆 Request 時，Apache 會為這個 Request 建立一個新的執行緒來處理，這樣的方式會造成每個 Request 都需要分配一個新的執行緒。因此，當同時有大量的 Request 時，會造成系統資源浪費。
- 而 Nginx 的 event-driven architecture 則是使用非同步的方式來處理 Request，當收到一筆 Request 時，Nginx 會將這個 Request 加入到 Event Queue 中，然後使用幾個固定數量 worker processes 來非同步的處理所有的 Request。因此，Nginx 總共的執行緒數量是固定的，這樣可以減少系統資源的浪費並且保持高效率。

Problem 1.2 (b)

- Master Process 會負責 listen client 的 requests，並且透過監控各個 worker process 的狀態來決定要將連線分配給哪一個 worker process。同時，Master Process 也會負責處理設定檔、以及決定是否要新增、刪除或重啟 worker process。
- Worker Process 會負責處理 client 的 requests，並且將處理的結果回傳給 client。每一個 worker process 都是獨立的一個執行緒，彼此之間不會互相影響。

Problem 1.2 (c) 為了要能在 1024 以下的 port 上監聽 client 的 requests，Nginx 的 Master Process 預設需要以 root 身份來啟動。而 worker process 則是以非 root 的使用者 www-data 來啟動。

1.3 憑證

References:

- <https://www.dottedsign.com/zh-tw/blog/product/pki>
- 公開金鑰基礎建設 - 維基百科
- 傳輸層安全性協定 - 維基百科
- 什麼是 ACME 協議，它是如何工作的？
- <https://docs.nginx.com/nginx/admin-guide/security-controls/terminating-ssl-http/>
- Passing Request Headers - Nginx
- What is SSL Termination?

Problem 1.3 (a) PKI 為公開金鑰基礎建設的簡稱，是一套完整的架構，用於建立與管理數位憑證。在此架構中，CA 負責簽發數位憑證，將使用者與其公鑰憑證建立信任關係，並驗證憑證的有效性與真實性。而 TLS 則是一種專門用於加密網路傳輸的協定，其安全性依賴於由 PKI 簽發的數位憑證。在 TLS 握手過程中，伺服器會將經過 CA 認證的數位憑證發送給對端，用以進行身份驗證，進而協商出安全且加密的通訊通道。

Problem 1.3 (b) ACME 是一種自動化的憑證管理協定，主要用於簡化數位憑證的申請、驗證、簽發和更新過程。ACME 協定的工作流程如下：

1. 客戶端向 CA 提出憑證簽發請求，並提供所需的域名資訊。
2. CA 會生成一個挑戰，要求客戶端證明其對該域名的控制權。常見的挑戰類型有許多種，以 Let's Encrypt 來說，挑戰的類型包含 HTTP-01 和 DNS-01。
3. 客戶端根據 CA 提供的挑戰資訊，進行相應的驗證操作，例如在指定的網址上放置特定的檔案，或是在 DNS 紀錄中加上指定的 TXT 紀錄。
4. CA 驗證客戶端的回應是否正確，若正確則簽發數位憑證。
5. 客戶端獲得數位憑證後，可以將其安裝到伺服器上，並開始使用 HTTPS 加密通訊。

目前常見的 ACME 客戶端有 Certbot、acme.sh 等等。

Problem 1.3 (c) 若以 IP 位址簽發憑證，一旦伺服器的 IP 位址發生變更，就得要重新簽憑證，這將導致管理上的不便。再者，DHCP、反向代理等技術或是伺服器配指的調整均可能引起 IP 位址的變動，進一步影響憑證的有效性。相較之下，網域名稱具有相對穩定的特性，且可透過 DNS 將其解析至正確的 IP 位址，從而避免因 IP 位址變動而導致憑證失效的問題。此外，在購買網域的過程中，使用者通常需要提供姓名、地址、電話等個人資訊，這些資料會被註冊商記錄於 WHOIS 資料庫中，進一步增強使用者身份的可信度。同時，使用者在辨識網站時多以網域名稱作為身份標識。因此，整體來說，使用網域名稱簽發憑證比用 IP 位址更好。

Problem 1.3 (d) SSL termination 指的是將 HTTPS 的連線在負載平衡器或反向代理伺服器解密，並且在內部改以 HTTP 的方式進行通訊。而在 Nginx 上，會透過反向代理時以以下指令新增標頭的方式來告訴內部的伺服器，原本的請求是 HTTPS 的連線：

```
1 proxy_set_header X-Forwarded-Proto $scheme;
```

2 Web Server Configurations

References:

- https://nginx.org/en/linux_packages.html#Debian
- ufw: 簡易防火牆設置
- Return custom 403 error page with nginx - Stack Overflow
- Including default files and folders in user's home directory - Server Fault
- How to Enable Userdir with Nginx
- Nginx : Enable Userdir
- nginx proxy_pass using subfolder - Server Fault
- 參考我先前架設的 Nginx 網頁 Reverse Proxy 設定
- https://nginx.org/en/docs/http/load_balancing.html
- 淺談 Nginx 基本配置、負載均衡、緩存和反向代理

2.1 Basic Setups

1. 下載 iso 檔案

2. 建立 qcow2 檔案

```
1 qemu-img create -f qcow2 debian.qcow2 16G
```

3. 使用 qemu 開啟 Debian 12.9 installer

```
1 qemu-system-x86_64 -enable-kvm \  
2   -cpu host \  
3   -m 8G \  
4   -cdrom debian-12.9.0-amd64-netinst.iso \  
5   -drive file=debian.qcow2,format=qcow2 \  
6   -boot d \  
7   -monitor stdio \  
8   -nic user,hostfwd=tcp::8890-:80,hostfwd=tcp::1102-:22 \  
9   -vnc :2100,password=on
```

4. 使用 VNC 連線上去之後，選擇 install.

5. 在安裝過程中，我使用 debian-hw6 作為 hostname，並且選擇 nasahw6.net 作為 domain name。之後，我創立一個 username 為學號的使用者。接著，我選擇安裝 gnome、ssh-server 和 standard system utilities 作為額外的套件。

6. 安裝完成後關機，我使用以下指令重開 VM：

```
1 qemu-system-x86_64 -enable-kvm \  
2   -cpu host \  
3   -m 8G \  
4   -drive file=debian.qcow2,format=qcow2 \  
5   -boot d \  
6   -monitor stdio \  
7   -nic user,hostfwd=tcp::8890-:80,hostfwd=tcp::1102-:22 \  
8   -vnc :2100,password=on
```

7. 使用 ssh 連線到 VM 上，並更新套件。

```
1 su -  
2 apt update  
3 apt upgrade  
4 apt install -y sudo vim git curl wget net-tools libpq-dev neofetch  
5 usermod -aG sudo b13901011
```

8. 安裝 Nginx 所需的套件。

```
1 apt install -y gnupg2 ca-certificates lsb-release debian-archive-keyring
```

9. 設定 Nginx 的 repository。

```
1 curl https://nginx.org/keys/nginx_signing.key | gpg --dearmor | sudo tee  
  ↪ /usr/share/keyrings/nginx-archive-keyring.gpg >/dev/null  
2 echo -e "Package: *\nPin: origin nginx.org\nPin: release  
  ↪ o=nginx\nPin-Priority: 900\n" | sudo tee /etc/apt/preferences.d/99nginx
```

10. 更新 repository 並安裝 Nginx。

```
1 sudo apt update  
2 sudo apt install -y nginx
```

11. 啟動 Nginx。

```
1 sudo systemctl start nginx  
2 sudo systemctl enable nginx
```

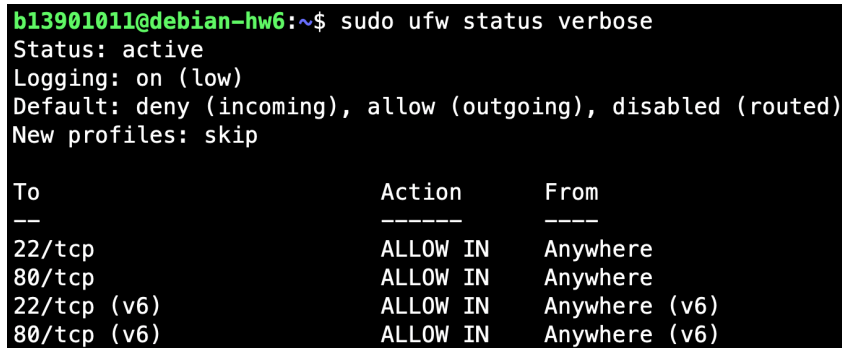
12. 安裝 ufw，並啟用

```
1 sudo apt install -y ufw
```

```

2 sudo ufw allow ssh
3 sudo ufw allow http
4 sudo ufw enable
5 sudo ufw status verbose

```



```

b13901011@debian-hw6:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
80/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)

```

Figure 1: UFW 設定

13. 為了驗證 ufw 的設定，我使用以下指令重開 VM，並且使用 ssh 連線到 VM 上。

```

1 qemu-system-x86_64 -enable-kvm \
2   -cpu host \
3   -m 8G \
4   -drive file=debian.qcow2,format=qcow2 \
5   -boot d \
6   -monitor stdio \
7   -nic user,hostfwd=tcp::8890-:80,hostfwd=tcp::1102-:22,hostfwd=tcp::8000-:8000 \
8   -vnc :2150,password=on

```

其中，VM 的 port 22 映射到我的工作站 port 1102，port 80 映射到我的工作站 port 8890 的 port 8000 映射到我的工作站 port 8000，並且改掉 vnc 的 port 為 2150 以避開衝突。

14. 我使用 `ssh localhost -p 1102` 成功的連線到 VM 上，代表 port 22 防火牆有設定好。
15. 在 VM 中使用以下指令，在 port 8000 架設一個簡單的 HTTP Server。

```

1 cd /usr/share/nginx/html
2 python3 -m http.server 8000

```

16. 回到工作站，使用以下指令，確認 port 80 和 port 8000 的防火牆有設定好。

```

1 curl --max-time 5 localhost:8890 # port 80 of VM is 8890 of workstation
2 curl --max-time 5 localhost:8000

```

```
[b13901011@nasa-ws1 ~]$ curl --max-time 5 localhost:8890
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Default Page</title>
  </head>
  <body>
    <p>b13901011: Default</p>
  </body>
</html>
[b13901011@nasa-ws1 ~]$ curl --max-time 5 localhost:8000
curl: (28) Operation timed out after 5002 milliseconds with 0 bytes received
[b13901011@nasa-ws1 ~]$
```

Figure 2: 確認 port 8000 和 port 80 (8890) 的防火牆有設定好

17. 我在我的電腦瀏覽器上，以 < 工作站 ip >:8890 的方式連線到 VM 上的 Nginx 網頁伺服器，其中 8890 是我映射到 VM port 80 的 port，可以看到 Nginx 的預設網頁。



Figure 3: Nginx 預設網頁

2.2 Default Page

1. 首先，我在 VM 上建立一個新的資料夾，並在裡面建立一個 index.html 檔案，內容如下:

```
1 sudo mkdir /var/www/
2 sudo mkdir /var/www/html/
3 vim /var/www/html/index.html

1 <!-- /var/www/html/index.html -->
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Default Page</title>
8 </head>
9 <body>
10  <p>b13901011: Default</p>
11 </body>
12 </html>
```


2. 接下來我修改 Nginx 的設定檔，將預設的網頁路徑改成我剛剛建立的資料夾。Nginx 的設定檔在 `/etc/nginx/conf.d/default.conf`。我找到 `location /` 的設定，修改如下：

```

1 location / {
2     root    /var/www/html;
3     index  index.html index.htm;
4 }

```

3. 接著，我重新啟動 Nginx 服務，並在工作站的瀏覽器上輸入 `< 工作站 ip>:8890`，可以看到我剛剛建立的網頁。

```

1 sudo systemctl restart nginx

```

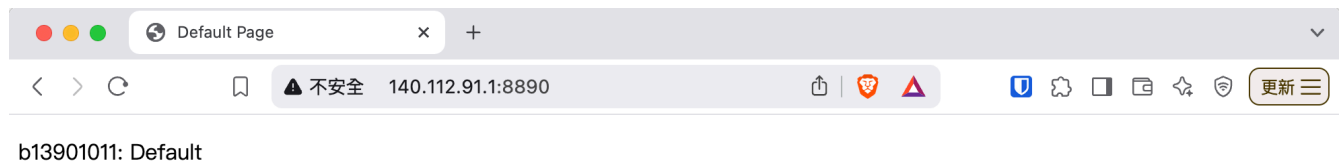


Figure 4: Edited Default Page

2.3 403 Forbidden

1. 建立 `forbidden` 的資料夾，並在裡面建立一個 `index.html` 檔案，內容如下：

```

1 sudo mkdir /var/www/html/forbidden
2 sudo echo "<p>index page in /forbidden, shouldn't see this.</p>" >
   ↪ /var/www/html/forbidden/index.html

```

2. 建立一個 403 Forbidden 的網頁，並將它放在 `/var/www/html/403.html`。指令如下：

```

1 sudo echo "<p>b13901011: 403 Forbidden</p>" > /var/www/html/403.html

```

3. 修改 Nginx 的設定檔，新增以下設定：

```

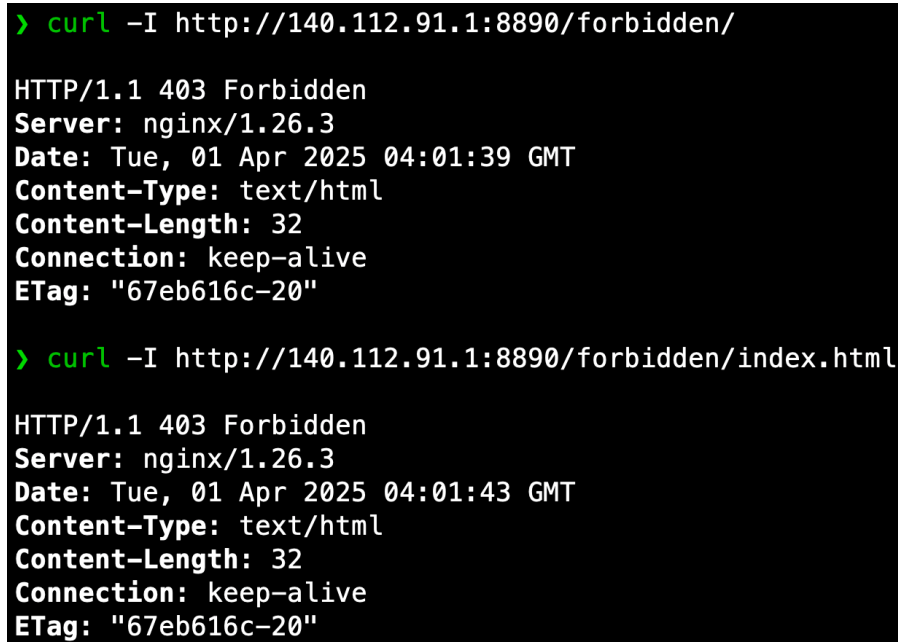
1 error_page 403 /403.html;
2 location = /403.html {
3     root /var/www/html;
4     allow all;
5 }
6 location /forbidden {
7     return 403;
8 }

```

4. 在我的筆電，使用以下指令測試 http 回傳碼：

```
1 curl -I http://140.112.91.1:8890/forbidden/  
2 curl -I http://140.112.91.1:8890/forbidden/index.html
```

由下圖結果中，可以看到回傳的 http status code 皆為 403 Forbidden，代表設定正確。



```
> curl -I http://140.112.91.1:8890/forbidden/  
  
HTTP/1.1 403 Forbidden  
Server: nginx/1.26.3  
Date: Tue, 01 Apr 2025 04:01:39 GMT  
Content-Type: text/html  
Content-Length: 32  
Connection: keep-alive  
ETag: "67eb616c-20"  
  
> curl -I http://140.112.91.1:8890/forbidden/index.html  
  
HTTP/1.1 403 Forbidden  
Server: nginx/1.26.3  
Date: Tue, 01 Apr 2025 04:01:43 GMT  
Content-Type: text/html  
Content-Length: 32  
Connection: keep-alive  
ETag: "67eb616c-20"
```

Figure 5: test http status code

5. 接著，我在瀏覽器上輸入 140.112.91.1:8890/forbidden，可以看到我建立的 403 頁面。

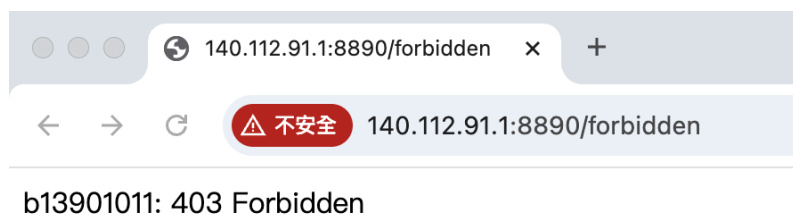


Figure 6: 403 Forbidden Page

2.4 User directory

1. 我可以編輯 /etc/skel 資料夾中的檔案，讓新建立的使用者可以擁有預設的檔案。這些檔案會在新使用者建立時自動複製到新使用者的家目錄中。

```
1 sudo mkdir /etc/skel/htdocs  
2 sudo chmod 755 /etc/skel/htdocs
```

2. 在 `/etc/nginx/conf.d/default.conf` 中，新增內容如下：

```
1 location ~ ^/^(.+)/?$ {
2     alias /home/$1/htdocs$2;
3     index index.html index.htm index.php;
4     autoindex on;
5 }
```

3. 重啟 Nginx 服務。

```
1 sudo systemctl restart nginx
```

4. 在 VM 上建立一個新的使用者：

```
1 sudo adduser user1
2 sudo chmod 711 /home/user1
```

建立完成後，可以發現裡面內建了 `htdocs` 資料夾，並且可以在裡面建立檔案。

```
root@debian-hw6:/home/user1# ls -al
total 32
drwx----- 3 user1 user1 4096 Apr  1 15:06 .
drwxr-xr-x 4 root  root  4096 Apr  1 15:06 ..
-rw-r--r-- 1 user1 user1  220 Apr  1 15:06 .bash_logout
-rw-r--r-- 1 user1 user1 3526 Apr  1 15:06 .bashrc
-rw-r--r-- 1 user1 user1 5290 Apr  1 15:06 .face
lrwxrwxrwx 1 user1 user1    5 Apr  1 15:06 .face.icon -> .face
drwxr-xr-x 2 user1 user1 4096 Apr  1 15:06 htdocs
-rw-r--r-- 1 user1 user1  807 Apr  1 15:06 .profile
root@debian-hw6:/home/user1#
```

Figure 7: UserDir

5. 在 `htdocs` 資料夾中建立一個 `index.html` 檔案，內容如下：

```
1 sudo echo "<p>user1: UserDir</p>" > /home/user1/htdocs/index.html
```

6. 使用瀏覽器，連上 `140.112.91.1:8890/ user1/`，可以看到我剛剛建立的網頁。

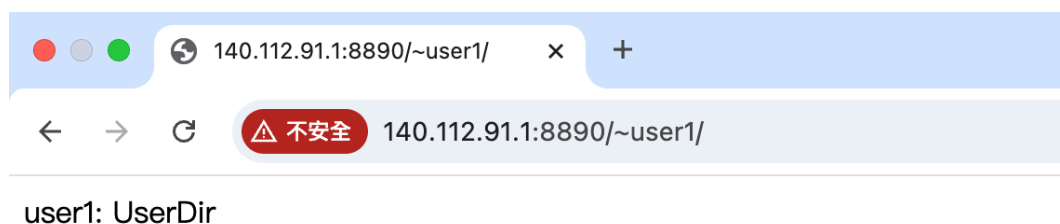


Figure 8: UserDir Page

2.5 Reverse Proxy

1. 首先，使用以下指令重新開啟 ServerMain VM：

```

1 qemu-system-x86_64 -enable-kvm -cpu host -m 8G \
2   -drive file=debian.qcow2,format=qcow2 \
3   -boot d \
4   -monitor stdio \
5   -nic user,hostfwd=tcp::8890-:80,hostfwd=tcp::1102-:22 \
6   -net nic,macaddr=52:54:00:00:00:01 -net vde \
7   -vnc :2100,password=on

```

2. 以 VNC 連線上去，並且打開 terminal，修改 `/etc/network/interface` 檔案，加入以下內容：

```

1 auto ens3
2 iface ens3 inet static
3     address 10.0.2.15/24
4     gateway 10.0.2.2

```

3. 使用以下指令重啟網路服務：

```

1 sudo systemctl restart networking

```

4. 建立 ServerA 和 ServerB，我使用先前 Problem 2.1 安裝完的 Debian qcow2 備份檔，複製兩份作為 ServerA 和 ServerB 的 VM，並且使用以下指令開啟 VM：

```

1 qemu-system-x86_64 -enable-kvm -cpu host -m 8G \
2   -drive file=serverA.qcow2,format=qcow2 \
3   -boot d \
4   -monitor stdio \
5   -nic user,hostfwd=tcp::8891-:80,hostfwd=tcp::1103-:22 \
6   -net nic,macaddr=52:54:00:00:00:AA -net vde \
7   -vnc :2200,password=on

```

```

1 qemu-system-x86_64 -enable-kvm -cpu host -m 8G \
2   -drive file=serverB.qcow2,format=qcow2 \
3   -boot d \
4   -monitor stdio \
5   -nic user,hostfwd=tcp::8892-:80,hostfwd=tcp::1104-:22 \
6   -net nic,macaddr=52:54:00:00:00:BB -net vde \
7   -vnc :2300,password=on

```

5. 對於 serverA 和 serverB，重複步驟 2.
6. 在三台機器上分別 ip a，得到其共通內網 ip 分別為：
 - ServerMain: 192.168.125.218
 - ServerA: 192.168.126.131
 - ServerB: 192.168.126.148

同時，我在 ServerA 和 ServerB 上使用 `curl 192.168.125.218`，可以看到 ServerMain 的網頁，代表它們可以互相連線。

7. 接著我在 ServerA 和 ServerB 參考 problem 2.1 的步驟安裝 Nginx 和 ufw，並且啟用。
8. 使用以下指令，在 ServerA 和 ServerB 上建立一個 index.html 檔案

```

1 # Server A
2 sudo mkdir -p /var/www/html/
3 sudo echo "<p>I am Server A</p>" > /var/www/html/index.html
4 # server B
5 sudo mkdir -p /var/www/html/
6 sudo echo "<p>I am Server B</p>" > /var/www/html/index.html

```

9. 修改 ServerA 和 ServerB 的 Nginx 設定檔，將預設的網頁路徑改成我剛剛建立的資料夾。Nginx 的設定檔在 `/etc/nginx/conf.d/default.conf`。我找到 `location /` 的設定，修改如下：

```

1 location / {
2     root    /var/www/html;
3     index  index.html index.htm;
4 }

```

接著，重啟 Nginx 服務。

```

1 sudo systemctl restart nginx

```

10. 我使用瀏覽器直接連線到 Server A 和 ServerB 的網頁，分別是 140.112.91.1:8891 和 140.112.91.1:8892，可以看到我剛剛建立的網頁。

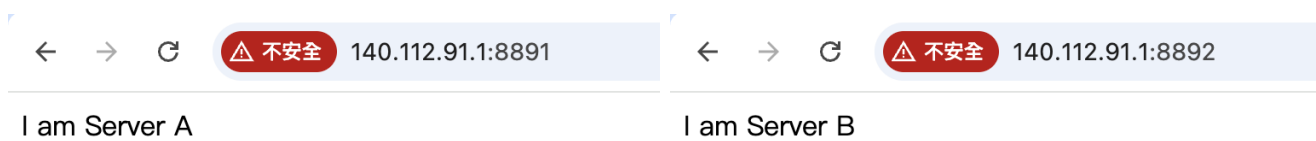


Figure 9: Test by directly connecting to Server A and Server B

11. 回到 ServerMain，設定反向代理，修改設定檔案 `/etc/nginx/conf.d/default.conf`，在最前面新增以下：

```

1 upstream serverA {
2     server 192.168.126.131:80;
3     keepalive 16;
4 }
5 upstream serverB {
6     server 192.168.126.148:80;
7     keepalive 16;
8 }

```

12. 接著，我在 `server` 區塊的設定中，加入以下內容：

```

1 location /serverA {
2     proxy_pass http://serverA/;
3     proxy_set_header Host $host;
4     proxy_set_header X-Real-IP $remote_addr;
5     proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
6     proxy_set_header X-Forwarded-Proto $scheme;
7 }
8
9 location /serverB {
10     proxy_pass http://serverB/;
11     proxy_set_header Host $host;
12     proxy_set_header X-Real-IP $remote_addr;
13     proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
14     proxy_set_header X-Forwarded-Proto $scheme;
15 }

```

13. 以 `sudo systemctl restart nginx` 重啟 Nginx 服務。
14. 從瀏覽器當中，分別輸入網址 `http://140.112.91.1:8890/serverA/` 和 `http://140.112.91.1:8890/serverB/`，可以看到 Server A 和 Server B 的網頁。

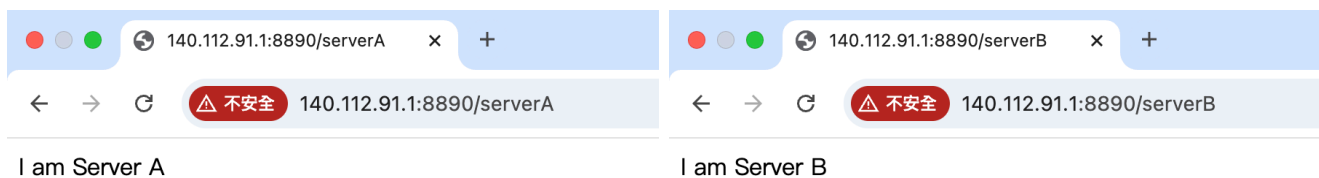


Figure 10: Result of Reverse Proxy

2.6 Load Balancing

- 將 Server B 的 qcow2 檔案複製一份，並命名為 serverC.qcow2。之後，再以以下指令開啟 VM：

```

1 qemu-system-x86_64 -enable-kvm -cpu host -m 8G \
2   -drive file=serverC.qcow2,format=qcow2 \
3   -boot d \
4   -monitor stdio \
5   -nic user,hostfwd=tcp::8893-:80,hostfwd=tcp::1105-:22 \
6   -net nic,macaddr=52:54:00:00:00:CC -net vde \
7   -vnc :2400,password=on

```

- 此檔案已經設定完網路，並且安裝和啟用過 Nginx 和 ufw，我使用 `ip a` 來獲 Server C 在 VDE 網路中的 IP 位址為 192.168.126.165。
- 接著，我去修改 Server C 的 `/var/www/html/index.html` 檔案，內容如下：

```

1 <p>I am Server C</p>

```

之後，並重新啟動 Nginx 服務。

```

1 sudo systemctl restart nginx

```

- 回到 ServerMain，修改 `/etc/nginx/conf.d/default.conf` 的設定檔，新增一條 upstream 如下：

```

1 upstream server {
2     server 192.168.126.131:80 max_fails=3 fail_timeout=30s;
3     server 192.168.126.148:80 max_fails=3 fail_timeout=30s;
4     server 192.168.126.165:80 backup;
5 }

```

- 接著，我在 `server` 區塊的設定中，加入以下內容：

```

1 location /balance {
2     proxy_pass http://server/;
3     proxy_set_header Host $host;
4     proxy_set_header X-Real-IP $remote_addr;
5     proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
6     proxy_set_header X-Forwarded-Proto $scheme;
7 }

```

- 以 `sudo systemctl restart nginx` 重啟 Nginx 服務。
- 完成以上之後，我在瀏覽器中以 `http://140.112.91.1:8890/balance/` 的方式連線到 ServerMain 上的 Nginx 網頁，可以看到 Server A 的網頁，並且重新整理之後可以看到 Server B 的網頁，可以確認成功設定 Round Robin 負載均衡。

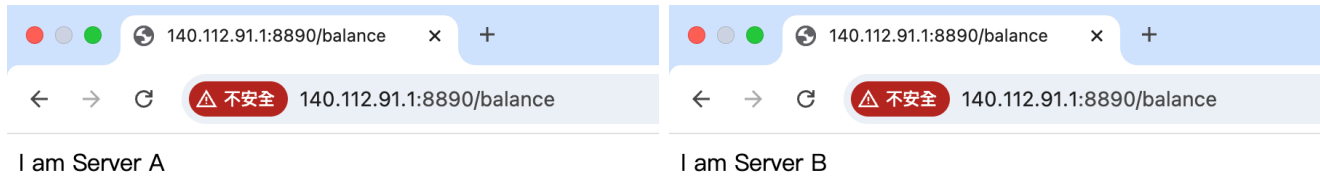


Figure 11: Load Balancing Page

- 我關閉 Server A 和 Server B，再重新整理網頁，等待 30 秒左右之後，會發現 Server C 的網頁會顯示出來。



Figure 12: Load Balancing Page

Appendix

以下附上我在 ServerMain 上的完整 Nginx 設定檔 `/etc/nginx/conf.d/default.conf` 的內容，包含了每一小題的設定：

```

1 upstream server {
2     server 192.168.126.131:80 max_fails=3 fail_timeout=30s;
3     server 192.168.126.148:80 max_fails=3 fail_timeout=30s;
4     server 192.168.126.165:80 backup;
5 }
6
7 upstream serverA {
8     server 192.168.126.131:80;
9     keepalive 16;
10 }
11

```



```
12 upstream serverB {
13     server 192.168.126.148:80;
14     keepalive 16;
15 }
16
17 server {
18     listen      80;
19     server_name localhost;
20
21     location / {
22         root    /var/www/html;
23         index   index.html index.htm;
24     }
25
26     location ~ ^/~(.*?)(/.*)?$ {
27         alias /home/$1/htdocs$2;
28         index  index.html index.htm;
29         autoindex on;
30     }
31
32     error_page 403 /403.html;
33     location = /403.html {
34         root    /var/www/html;
35         allow all;
36     }
37     location /forbidden {
38         return 403;
39     }
40
41     location /serverA {
42         proxy_pass http://serverA/;
43         proxy_set_header Host $host;
44         proxy_set_header X-Real-IP $remote_addr;
45         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
46         proxy_set_header X-Forwarded-Proto $scheme;
47     }
48     location /serverB {
49         proxy_pass http://serverB/;
50         proxy_set_header Host $host;
51         proxy_set_header X-Real-IP $remote_addr;
52         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
53         proxy_set_header X-Forwarded-Proto $scheme;
54     }
```

```
55
56     location /balance {
57         proxy_pass http://server/;
58         proxy_set_header Host $host;
59         proxy_set_header X-Real-IP $remote_addr;
60         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
61         proxy_set_header X-Forwarded-Proto $scheme;
62     }
63
64     # redirect server error pages to the static page /50x.html
65     #
66     error_page 500 502 503 504 /50x.html;
67     location = /50x.html {
68         root /usr/share/nginx/html;
69     }
70 }
```
